FINFLOW · GUIA DE PUBLICAÇÃO

DA MÁQUINA LOCAL À WEB

Seu roadmap de produção, com segurança em cada etapa.

Um roteiro prático para publicar frontend, API e banco de dados em serviços separados — com CORS, HTTPS, TLS, segredos, migrações e uma verificação final.

Adaptado à arquitetura atual do FinFlow

Revisado em 03 nov 2026

Imprimir / salvar como PDF

ARQUITETURA SUGERIDA

UI	React + Vite	Vercel
API	Node + Express	Render
DB	MySQL-compatible	TiDB

Recomendação honesta: valide primeiro com dados fictícios

O plano gratuito do Render é explicitamente destinado a testes/hobby, pode suspender serviços ociosos e não oferece disco persistente. O TiDB Starter também tem cotas e pode bloquear novas conexões ao excedê-las. Para dados financeiros reais, migre para planos com SLA, backups verificáveis e monitoramento antes de convidar usuários.

01 Onde hospedar cada parte

Separar os serviços facilita deploys, mas exige configurar as URLs e a confiança entre eles.

FRONTEND · GRATUITO

V Vercel Hobby

React + Vite / arquivos estáticos

Conecta ao GitHub e publica a pasta `frontend`. Fornece domínio `vercel.app` e TLS/HTTPS gerenciado.

Ver plano e cotas ↗

API · ENSAIO GRÁTIS

R Render Free Web Service

Node.js + Express

Deploy do diretório `backend`; recebe `PORT` automaticamente. Pode "dormir" após inatividade e demorar a voltar.

Ver limites do gratuito ↗

BANCO · COTA GRATUITA

T TiDB Cloud Starter

Compatível com protocolo MySQL

Alternativa MySQL gerenciada. Exige conexão TLS; tem cota mensal, sem equivaler a uma garantia de backup/produção.

Ver plano e cotas ↗

Outra opção para arquivos estáticos: Render Static Site. Revise termos, disponibilidade regional, limites e eventual exigência de cartão antes de cadastrar o projeto; provedores alteram seus planos.

02 O que "grátis" significa na prática

Planeje as limitações antes que elas virem indisponibilidade ou perda de dados.

- ⏱ **Render Free:** serviço web pode dormir após 15 minutos sem tráfego; a próxima chamada pode aguardar cerca de um minuto.
- 💾 **Disco:** o filesystem do serviço Render é efêmero. Não use SQLite, arquivos locais ou uploads como armazenamento permanente.
- **Render Postgres gratuito:** não serve como banco permanente; a documentação atual informa expiração após 30 dias. Não é compatível com o MySQL que este app usa.
- **TiDB Starter MySQL:** a documentação informa 5 GiB de armazenamento e 50 milhões de RUs/mês por instância elegível; ao atingir a cota, novas conexões podem ser negadas até o reset/upgrade.
- 🔁 **Rotinas:** o backend executa liquidação de parcelas por timer. Um serviço gratuito que dorme não executa tarefas de forma contínua; teste o fluxo e não dependa de horário exato.
- **Backups:** exporte e teste a restauração. Não assuma retenção, backup automático ou SLA no free tier.

03

Preparação no repositório

Faça isso antes de conectar serviços públicos ao banco.

1 Confirme o estado local

Rode frontend e backend localmente, valide login, setup inicial, criação/edição de transação, dívidas, investimentos, importação e exportação.

2 Confira o ponto de partida

- Frontend: `frontend`, comando de build `npm run build`, saída `dist`.
- API: `backend`, inicia com `npm start`, escuta `PORT` e `0.0.0.0`.
- O backend lê variáveis do ambiente e, localmente, o arquivo `.env` na raiz do repositório.
- As migrações ficam em `backend/src/migrations`; o script existente é `npm run db:migrate`.

3 Prepare o Git com segurança

Faça push para um repositório privado, se preferir. Verifique que `.env`, senhas, dumps do banco e chaves JWT não estão commitados. Nunca use variáveis `VITE_*` para segredos: elas podem ser incorporadas no JavaScript público.

04

Crie o banco e prepare TLS

O código atual usa Sequelize + mysql2; o TiDB exige conexão criptografada.

1

Crie uma instância MySQL-compatible

No console TiDB Cloud, crie TiDB Cloud Starter em uma região próxima do serviço da API. Salve host, usuário com prefixo da instância, senha e nome do banco. Crie o banco no console, se necessário, e limite o usuário às permissões necessárias.

2

Habilite TLS no Sequelize

Antes de migrar, ajuste `backend/src/config/config.js`. Mantenha a verificação de certificado ligada; não use `rejectUnauthorized: false`.

```
const config = {
  username: process.env.MYSQL_USER,
  password: process.env.MYSQL_PASSWORD,
  database: process.env.MYSQL_DATABASE,
  host: process.env.MYSQL_HOST,
  dialect: "mysql",
  dialectOptions: {
    ssl: {
      minVersion: "TLSv1.2",
      rejectUnauthorized: true
    }
  }
};
```

3

Habilite TLS também no script de migração

Em `backend/scripts/db-migrate.js`, passe TLS na conexão `mysql2` usada para criar/verificar o banco:

```
const connection = await mysql.createConnection({
  host: process.env.MYSQL_HOST,
  user: process.env.MYSQL_USER,
  password: process.env.MYSQL_PASSWORD,
  ssl: {
    minVersion: "TLSv1.2",
    rejectUnauthorized: true
  }
});
```

Confirme na tela “Connect” do TiDB o hostname, usuário e os parâmetros atualizados. O usuário normalmente inclui o prefixo de instância; copiar somente “root” pode falhar.

4

Migre e confira o schema

Faça primeiro um backup/export. Configure temporariamente no seu terminal seguro `MYSQL_HOST`, `MYSQL_USER`, `MYSQL_PASSWORD` e `MYSQL_DATABASE` da instância de teste; na pasta `backend`, rode `npm.cmd run db:migrate`. O script cria o banco caso ainda não exista e aplica as migrations.

> Teste todas as migrations neste provedor antes de importar dados reais: compatibilidade de protocolo MySQL não garante compatibilidade idêntica de cada operação SQL. Não rode `db:drop` contra produção.

5 Crie o administrador e migre seus dados, se houver

Com schema vazio, use a tela `/setup` e confirme em `/auth/setup-status` que a configuração inicial foi concluída. Se já tem dados locais, exporte o backup completo pelo app, teste a restauração numa instância de ensaio e só então importe uma vez no ambiente final, conferindo transações, categorias, dívidas/parcelas e investimentos/movimentos relacionados.

> A configuração inicial é um endpoint público de uso único: não compartilhe a URL do app/API antes de criar o administrador inicial. CORS não impede chamadas diretas de scripts ou servidores.

> O TiDB documenta TLS obrigatório no Starter. A API e o comando de migração precisam usar TLS; configurar apenas HTTPS no navegador não protege a conexão backend↔banco.

05

Publique a API: Node + Express no Render

Crie um Web Service conectado ao repositório, selecionando como raiz o diretório `backend`.

1

Configure build, start e health check

- Runtime: Node. Root Directory: `backend`.
- Build Command: `npm install`. Start Command: `npm start`.
- Health Check Path: `/health`. O endpoint existente confirma que o processo HTTP responde; não testa a conexão ao banco.
- Não fixe uma porta em código/plataforma: o servidor usa `process.env.PORT` e `0.0.0.0`.

2

Cadastre variáveis no painel da API

Use a área Environment do serviço. Valores são placeholders; os reais devem vir do console do banco e ser gerados por você.

VARIÁVEL	VALOR NO SERVIÇO BACKEND	NOTA
`NODE_ENV`	`production`	Ativa configuração estrita de CORS.
`JWT_SECRET`	Segredo aleatório exclusivo, longo	Não reutilize o exemplo/local nem exponha no frontend.
`MYSQL_HOST`	Hostname de conexão do TiDB	Sem `https://`.
`MYSQL_USER`	Usuário completo fornecido pelo TiDB	Inclua o prefixo do cluster.
`MYSQL_PASSWORD`	Senha do usuário de aplicação	Secret do provedor, não Git.
`MYSQL_DATABASE`	Nome exato do banco	Schema já migrado.
`FRONTEND_ORIGINS`	`https://seu-app.vercel.app`	Origem exata, sem barra final; origens adicionais separadas por vírgula.
`TRUST_PROXY`	`1`	Somente se API estiver atrás do proxy gerenciado do host; o app usa um salto confiável.
`PORT`	Deixe o provedor definir	Render injeta a porta pública interna.

> Gere o JWT secret localmente com Node: `node -e "console.log(require('crypto').randomBytes(48).toString('base64url'))"`. Copie o resultado diretamente para o secret do Render; não salve em arquivo versionado.

3

Entenda CORS e HTTPS

O código lê `FRONTEND_ORIGINS` e compara a origem exata. Quando `NODE_ENV=production`, não há origem local liberada por padrão. O HTTPS é terminado pelo Render; use somente a URL `https://…onrender.com` no frontend. Se conectar domínio próprio, cadastre também a nova origem exata no CORS.

- Não use `*` como origem em produção.
- Não coloque caminho (`/home`) ou barra final no valor da origem.
- URLs de Preview da Vercel não são automaticamente permitidas; mantenha produção e Preview controlados.
- Não é necessário habilitar cookies cross-site para a autenticação Bearer deste app.

4 Faça o primeiro deploy e anote a URL pública

Após deploy bem-sucedido, teste `https://SUA-API.onrender.com/health`. Copie somente o hostname/URL HTTPS; será usado em `VITE_API_URL`.

06 Publique o React + Vite na Vercel

É um frontend estático: a URL da API é compilada durante o build.

1 Crie o projeto Vercel a partir do Git

- Selecione o repositório e defina Root Directory como `frontend`.
- Framework Preset: Vite (ou detecção automática).
- Build Command: `npm run build`; Output Directory: `dist`.
- Cadastre `VITE_API_URL=https://SUA-API.onrender.com` no ambiente Production.

> `VITE_API_URL` não é segredo; será público no bundle. Nunca cadastre JWT secret, senha SQL ou credenciais na Vercel.

2 Configure navegação SPA (React Router)

Este app tem caminhos como `/home` e `/setup`. Para abrir ou atualizar uma rota diretamente na Vercel, adicione `frontend/vercel.json` com fallback para o index da SPA:

```
{
  "$schema": "https://openapi.vercel.sh/vercel.json",
  "rewrites": [
    { "source": "/(.*)", "destination": "/index.html" }
  ]
}
```

3 Conecte a origem ao CORS e redeploy

Copie a URL de produção da Vercel e atualize `FRONTEND_ORIGINS` no Render, sem barra final. Salve, deixe o backend redeployar e faça novo deploy do frontend se alterou `VITE_API_URL`. Vercel e Render fornecem TLS gerenciado nos seus domínios.

07

Validação de ponta a ponta

Teste a publicação como se fosse uma pessoa usuária, sem dados reais inicialmente.

- [ ] A URL pública da API responde em `/health` usando HTTPS.
- [ ] A URL do frontend abre e o certificado do navegador é válido.
- [ ] Atualizar diretamente `/login`, `/home` e `/setup` não dá 404.
- [ ] Setup inicial cria o administrador; confirme que não pode ser repetido após concluído.
- [ ] Login, sessão, criação e edição de transação funcionam após reload.
- [ ] CORS aceita o domínio oficial e rejeita uma origem desconhecida.
- [ ] Uma operação de escrita persiste no banco após novo deploy/restart da API.
- [ ] Importação, dívidas, parcelas, investimentos, exportação e recuperação de senha foram testados.
- [ ] Valores e datas importados foram conferidos no banco e na tela.
- [ ] Backup de teste foi exportado e uma restauração foi comprovada em instância separada.

Os quadrinhos ajudam na revisão visual; não são enviados nem persistidos por este arquivo.

08

Antes de colocar dados reais

HTTPS é necessário, mas não basta para tornar um serviço financeiro pronto para produção.

1

Troque o ambiente de ensaio por infraestrutura compatível com o risco

Use backend sempre ligado, banco gerenciado com backups automáticos/retidos, restauração point-in-time quando disponível, alertas, cotas e uma política de recuperação testada. Avalie controles contratuais, residência dos dados, disponibilidade, suporte e requisitos legais aplicáveis.

2

Operação e segredos

- Use credencial de banco exclusiva para a aplicação; evite usuário root e permissões globais.
- Ative MFA nas contas de hospedagem/Git; restrinja colaboradores e tokens.
- Gire credenciais se forem expostas; não envie dados financeiros ou tokens para logs.
- Monitore falhas de login, erros 5xx, conexões DB e limites de uso; defina alertas de custo.
- Faça atualizações de dependências e mantenha produção e Preview isolados.

3

Revise tarefas agendadas e escala

O servidor executa liquidação automática por `setInterval`. Serviços gratuitos podem pausar; múltiplas réplicas também podem executar a rotina ao mesmo tempo. Antes de escalar, transforme a tarefa em job idempotente/único e use scheduler/worker apropriado.

> Não importe backup financeiro pessoal para uma infraestrutura de ensaio sem entender retenção, acesso e exclusão. Use dataset fictício até ter controles e restauração verificados.

↗

Fontes oficiais e leitura adicional

Limites e condições consultados em 03/11/2026; confirme as páginas antes do deploy.

Vercel · Vite
Build, deploy e variáveis de ambiente

Vercel · planos e cotas
Hobby e uso incluído

Render · limitações gratuitas
Sleep, filesystem efêmero e expiração do Postgres free

Render · health checks
Configuração de endpoint de saúde

TiDB · conexão TLS
Requisito TLS e autoridades certificadoras

Vercel · configuração e rewrites
Fallback de SPA com React Router

Render · Web Services
Root, comandos, porta e variáveis

Render · TLS gerenciado
HTTPS e renovação automática

TiDB · planos Starter
Compatibilidade MySQL e cotas publicadas